VIETNAM GENERAL CONFEDERATION OF LABOR
TON DUC THANG UNIVERSITY
FACULTY OF INFORMATION TECHNOLOGY

NGUYEN THAI KHANH NAM – 523H0159
TRAN VAN QUI – 523H0174
LE KHAC THANH TRI – 523H0186

FINAL REPORT

DEEP LEARNING

HO CHI MINH CITY, 2026

VIETNAM GENERAL CONFEDERATION OF LABOR
TON DUC THANG UNIVERSITY
FACULTY OF INFORMATION TECHNOLOGY

NGUYEN THAI KHANH NAM – 523H0159
TRAN VAN QUI – 523H0174
LE KHAC THANH TRI – 523H0186

FINAL REPORT

DEEP LEARNING

Advised by
Dr. Le Anh Cuong

HO CHI MINH CITY, 2026

ACKNOWLEDGEMENT

Initially I would like to express my sincere gratitude to Ton Duc Thang University for creating an appropriate environment for self-development and studying. Faculty of Information Technology for supporting and assisting students during school. Furthermore, I am sincerely grateful for all instructors in my major, specifically Computer Science. Finally, I would like to express my sincere gratitude to Dr. Le Anh Cuong for his valuable guidance and support throughout this midterm report. He has been very helpful and patient in providing me with constructive feedback and suggestions to improve my work. I have accumulated deep insights about digital image processing and other related knowledge. I am honored and privileged to have him as my teacher and supervisor.

Ho Chi Minh city, 05th May 2026.

Author

(Signature and full name)

Nam

Nguyen Thai Khanh Nam

Qui

Tran Van Qui

Tri

Le Khac Thanh Tri

DECLARATION OF AUTHORSHIP

We hereby declare that this is our own project and is guided by Dr. Le Anh Cuong; The content research and results contained herein are central and have not been published in any form before. The data in the tables for analysis, comments and evaluation are collected by the main author from different sources, which are clearly stated in the reference section.

In addition, the project also uses some comments, assessments as well as data of other authors, other organizations with citations and annotated sources.

If something wrong happens, we'll take full responsibility for the content of my project. Ton Duc Thang University is not related to the infringing rights, the copyrights that We give during the implementation process (if any).

Ho Chi Minh city, 05th May 2026.

Author

(Signature and full name)

Nam

Nguyen Thai Khanh Nam

Qui

Tran Van Qui

Tri

Le Khac Thanh Tri

TABLE OF CONTENT

CHAPTER 1. INTRODUCTION

1.1 Background and Motivation

Visual Question Answering (VQA) is an integrated problem between Computer Vision and Natural Language Processing. A VQA model takes as input a pair (image, question) and must generate a reasonable answer based on the visual content of the image. Modern VQA systems have achieved high accuracy on English benchmarks such as VQA v2, GQA, and OK-VQA. However, for Vietnamese — a tonal language with unique grammatical structure — the volume of research remains very limited.

Vietnamese cuisine is a particularly suitable data domain for building Vietnamese VQA: (i) a rich variety of dishes with clear visual characteristics; (ii) abundant image data with publicly available datasets on Hugging Face; (iii) high practical applicability in tourism, food, and cultural education; (iv) this is an area where English pretrained models (such as BLIP-2) lack domain knowledge — creating an opportunity for fine-tuning to deliver real value.

1.2 Problem Statement

Our team defines the Vietnamese VQA problem for the culinary domain as follows:

Input: An image I containing a Vietnamese dish, along with a Vietnamese question Q related to the dish in the image.

Output: A Vietnamese answer A, concise (≤ 10 words), with content that is appropriate to both the image and the question.

The types of questions supported include: (1) Yes/No (“Does this dish have broth?”), (2) Recognition (“What dish is this?”), (3) Attribute (“What color is the broth?”), (4) Spatial (“Where is the cake placed?”), (5) Counting (“How many bowls of dipping sauce are there?”), and (6) other open-ended questions.

1.3 Objectives and Contributions of the Report

This project pursues four main objectives:

- Objective 1: Build a Vietnamese VQA dataset for the Vietnamese culinary domain of sufficient scale (≥ 2000 QA pairs) and quality.

- Objective 2: Implement and compare two architectural approaches — Approach A (modular architecture with CNN/ViT + LSTM/PhoBERT + Decoder) and Approach B (Multimodal pretrained such as BLIP-2).
- Objective 3: Conduct a detailed comparison between LSTM Decoder (configuration A1) and Transformer Decoder (configuration A2) on the same image+text encoder, to clarify the effect of the decoder type.
- Objective 4: Study and apply diverse VQA evaluation methods including exact-match accuracy, BLEU, ROUGE-L, METEOR, BERTScore, and LLM-as-a-judge.

In addition, the project also includes Part 2: Plant Disease Detection using Transfer Learning with EfficientNetB0 — a real-world agricultural application.

Main contribution: (1) Building a Vietnamese VQA dataset with 2,100 images × 30 dishes × 3 questions per image = 6,303 QA pairs, automatically generated by Gemini 2.5 Flash. (2) Implementing four model configurations ranging from classical architecture (LSTM) to Transformer and Multimodal Pretrained models. (3) Conducting an in-depth analysis of the differences between LSTM and Transformer decoders. (4) Presenting both the limitations and the lessons learned honestly throughout the project.

CHAPTER 2. THEORETICAL BACKGROUND

This chapter briefly presents the foundational knowledge blocks used in the project. The purpose is to help readers follow the subsequent chapters without needing to consult other references, while also clarifying why we chose specific technical components.

2.1 CNN and Vision Transformer (ViT)

Convolutional Neural Network (CNN) is the classical architecture for image processing, with convolution features that allow extraction of spatially invariant features (translation invariance) and reduce the number of parameters compared to fully connected networks. Famous CNN architectures include VGG, ResNet, and EfficientNet.

Vision Transformer (ViT) is a modern alternative, proposed by Dosovitskiy et al. (2020). ViT divides an image into small patches (typically 16×16 pixels), embeds each patch as a token, and then applies the Transformer Encoder architecture. The main advantage of ViT is its ability to capture long-range dependencies across the entire image through the self-attention mechanism. In the project, we use ViT-B/16 pretrained on ImageNet as the Image Encoder for both configurations A1 and A2.

2.2 LSTM and Transformer

Long Short-Term Memory (LSTM) is a variant of RNN, designed to overcome the vanishing gradient problem in traditional recurrent networks. LSTM uses gates — input gate, forget gate, output gate — to control the flow of information over time. Although surpassed by Transformer on many benchmarks, LSTM remains a reasonable choice in compact systems due to its lower computational cost and good performance on short sequences.

Transformer (Vaswani et al., 2017) is an architecture based entirely on attention mechanisms and does not require recurrence. The self-attention block allows each token to “see” all other tokens in the sequence with a cost of $O(n^2)$, instead of processing sequentially like an RNN. The Transformer Decoder uses masked self-attention to ensure causality during sequence generation, combined with cross-attention to exploit information from the encoder. This is the architecture used by most large language models today (GPT, T5, BERT, LLaMA, etc.).

In the project, we implement both types of decoders to directly compare answer generation capability: configuration A1 uses LSTM Decoder, configuration A2 uses Transformer Decoder, while keeping the same image encoder (ViT) and text encoder (PhoBERT).

2.3 PhoBERT – Vietnamese Language Model

PhoBERT (VinAI Research, 2020) is a RoBERTa-based model pretrained on 20GB of Vietnamese text from Wikipedia and news articles. It is currently one of the strongest Vietnamese language models across many benchmarks (NER, POS, Sentiment Analysis). In VQA, PhoBERT serves as the Text Encoder: it tokenizes Vietnamese questions and produces context-aware semantic representations for fusion with image features. The version used in this project is phobert-base with 135 million parameters.

2.4 BLIP-2 and Q-Former

BLIP-2 (Bootstrapping Language-Image Pre-training, Li et al., 2023) is a significant advancement in vision-language pretrained models. The BLIP-2 architecture consists of three components: (1) a frozen Image Encoder (e.g., ViT-G), (2) Q-Former — a lightweight Transformer module using learned query tokens to bridge images and language, (3) a frozen Large Language Model (LLM) (such as OPT or FlanT5). This approach allows leveraging powerful LLMs without retraining from scratch — only training the Q-Former (~188M parameters) is sufficient to create an effective bridge.

In the project, we use Salesforce/blip2-opt-2.7b — the version with ~3.7 billion parameters total — for both configuration B1 (zero-shot) and B2 (fine-tuned).

2.5 LoRA – Low-Rank Adaptation

Low-Rank Adaptation (Hu et al., 2021) is an efficient fine-tuning technique for large models. The core idea: instead of updating the entire weight matrix $W \in \mathbb{R}^{(d\times d)}$, LoRA decomposes the update into the product of two low-rank matrices: $\Delta W = BA$, where $B \in \mathbb{R}^{(d\times r)}$, $A \in \mathbb{R}^{(r\times d)}$, $r \ll d$. When r=16, the number of trainable parameters is reduced from d^2 to $2dr$ — saving significant VRAM and time. Importantly, after training is complete, LoRA can be merged back into the original model, causing no increase in inference latency.

Configuration B2 in the project uses LoRA with r=16, alpha=32, dropout=0.05, injected into the q_proj and v_proj matrices of OPT-2.7B. The trainable parameter

ratio is only ~0.13% (5.24M / 3.75B) — suitable for the Tesla T4 GPU (15.6 GB VRAM) on Google Colab.

CHAPTER 3. DATA

3.1 Data Source and Specialized Domain

We chose the Vietnamese cuisine data domain, using the publicly available vietnamese_food_images dataset on Hugging Face Hub contributed by TuyenTrungLe. The original dataset contains thousands of images of various Vietnamese dishes, pre-labeled at the dish class level.

To ensure balance and effective training with free Colab resources, we applied a stratified sampling strategy: selecting the first 30 dishes in the original dataset, taking 70 images per dish, for a total of 2,100 images. The dishes include: Bánh bèo, Bánh khọt, Bún bò Huế, Bún riêu, Bún đậu mắm tôm, Phở, Bánh mì, Cơm tấm, and many other traditional dishes — creating a diverse image distribution in terms of color, shape, and presentation.

3.2 QA Generation Process Using Gemini 2.5 Flash

The most difficult part of building the VQA dataset is generating (question, answer) pairs for each image. Manually labeling 2,100 images × 3 questions = 6,300 QA pairs is not feasible within a semester project timeline. We used Google's Gemini 2.5 Flash as a "virtual annotator" to generate the data.

Process: For each image, we send both the image and a Vietnamese prompt asking Gemini to generate 3 diverse QA pairs (yes/no, ingredients, color, recognition, attribute, spatial), with extremely short answers (≤ 10 words). The output is required to be valid JSON for easy parsing. The pipeline includes retry handling when rate limits occur (HTTP 429) and a resume mechanism from an old CSV file if the process is interrupted.

After generation, we randomly check samples to ensure quality. The majority of questions/answers generated by Gemini are of good quality, closely reflecting the image content and diverse in question types. This is a practical approach widely used in recent Multimodal research — known as synthetic data generation.

3.3 Data Analysis and Statistics

Table 3-1: Overview statistics of the Vietnamese Cuisine VQA dataset.

Criteria	Value
Data source	HuggingFace: TuyenTrungLe/vietnamese_food_images
Number of dish classes	30 dishes
Images per class	70 images
Total images	2,100 images
Questions per image	3 questions (generated by Gemini 2.5 Flash)
Total QA pairs	6,303 pairs (cleaned)
Answer length	≤ 10 words
Question types	Yes/No, Recognition, Color, Spatial, Counting, Other
Language	Vietnamese (both Q and A)

Vocabulary: After tokenizing all answers (lowercased, special punctuation removed), and applying a minimum frequency threshold of freq_threshold = 2, the vocabulary contains 693 words in total — including the 4 special tokens: <PAD>, <SOS>, <EOS>, <UNK>. This size is suitable for the specialized domain of food dishes.

Question type distribution (based on keyword classification): Yes/No accounts for the largest share (~60–65%), followed by Recognition (~21%), Color (~7%), Spatial (~3%), and other types (~4%). This distribution reflects the natural questions users usually ask when viewing a food image — they often ask to confirm the dish, visual characteristics, and attributes.

3.4 Dataset Splitting – Transparent Disclosure

This is a point we want to present honestly for readers to properly evaluate. The assignment requires splitting the dataset in an 80/10/10 ratio for train/val/test. During implementation, we encountered inconsistencies between notebooks:

- Notebooks A1 and A2 use train_test_split with test_size=0.2, i.e., an 80/20 split (5042 train / 1261 val), with no separate test set. The metrics for A1, A2 are calculated on the validation set.
- Notebooks B1 and B2 use the correct 80/10/10 ratio (5041 train / 631 val / 631 test). The metrics for B1, B2 are calculated on the test set.

The consequence of this inconsistency is that the results of A1/A2 and B1/B2 cannot be directly compared in a mathematically strict sense — because they are evaluated on different data subsets (val 1261 samples vs test 631 samples). However, since data was randomly split with the same random_state=42 and similar distributions, general comparative trends remain meaningful as a reference.

One more limitation that we acknowledge honestly: the test set was split randomly at the (image, question, answer) level rather than at the image level. This means the same image may appear in both the train and test sets with different questions. The assignment requires "≥ 50 manually prepared test samples, with no overlap between test and train images" — this requirement has not been fully satisfied. A future improvement would be to split by image (group-aware split) and add an independent manually collected test set.

Nevertheless, we decided to keep the current results in this report rather than adjusting the figures — because methodological honesty is more important than painting results in a favorable light. The metrics should be understood in this context.

CHAPTER 4. METHODOLOGY – APPROACH A (MODULAR ARCHITECTURE)

4.1 Architecture Overview

The task's Approach A requires building a modular architecture from basic components: Image Encoder, Text Encoder, Fusion module, and Answer Decoder. This "from-scratch" approach helps us understand the mechanism of a classical VQA model in depth. Configurations A1 and A2 share the same Image Encoder and Text Encoder, differing only in the Decoder — allowing a direct comparison of decoder type.

General data flow diagram of Approach A:

Image I → ViT-B/16 → image features (197 × 768)

Question Q → PhoBERT-base → question features (50 × 768)

Image features ⊕ Question features → Fusion (concat / element-wise)

Fusion → Decoder (LSTM in A1, Transformer in A2) → answer

4.2 Image Encoder and Text Encoder

Image Encoder – Vision Transformer (ViT-B/16): Each image is resized to 224×224, divided into 196 patches of 16×16 (plus 1 [CLS] token = 197 tokens), each patch embedded into a 768-dimensional vector, then passed through 12 Transformer Encoder layers. The output is a tensor of size (197 × 768) — containing both global information (CLS token) and spatial information (patch tokens) — very useful for the subsequent Spatial Attention mechanism.

Text Encoder – PhoBERT-base: Vietnamese questions are tokenized using the PhoBERT tokenizer (BPE-based, maximum 50 tokens per question), then passed through 12 pretrained RoBERTa layers. The output is a tensor (50 × 768), where the first vector is [CLS] representing the full semantic meaning of the question.

Fusion: The [CLS] vector from PhoBERT (768 dimensions) is used as the semantic context of the question, and is combined with the image patch features through the Spatial Attention mechanism (detailed in A1) or cross-attention (in A2). This is the key component that enables the model to "focus" on the image regions relevant to each question.

4.3 A1 – LSTM Decoder + Spatial Attention

Configuration A1 uses a 2-layer LSTM as decoder, with a Spatial Attention mechanism to select relevant image regions at each word generation step. Spatial Attention is implemented as follows:

• att1 = Linear_enc(image_features) — projects image features to attention space

• att2 = Linear_dec(hidden_state) — projects the current LSTM hidden state

• α = softmax(Linear_full(tanh(att1 + att2)))

• context = Σ α_i × image_features_i

At each timestep, the LSTM receives as input [embedding(previous_token); context_vector; question_embedding], updates the hidden state, and produces a probability distribution over the 693-word vocabulary. The process repeats until the <EOS> token is generated or max_len = 15 is reached.

Detailed hyperparameter table for A1:

Table 4-1: Detailed configuration of model A1.

Hyperparameter	Value
Image Encoder	ViT-B/16 pretrained ImageNet
Text Encoder	PhoBERT-base (135M params)
Decoder	LSTM 2 layers + Spatial Attention
Embedding size	512
Hidden size	1024
Number of epochs	30
Batch size	16

Hyperparameter	Value
Optimizer	AdamW
Learning rate	2e-5 (cosine schedule with warmup)
Loss function	Label-smoothed NLL Loss (ε = 0.1)
Inference strategy	Diverse Beam Search (3 nhóm × 2 beam)
Max answer length	15 tokens

Loss function: We use Label-smoothed NLL Loss with ε = 0.1 instead of plain Cross-Entropy. Label smoothing helps prevent the model from becoming overconfident in a single label, improves calibration, and often boosts generalization. Padding tokens are ignored in the loss using ignore_index=0.

Inference – Diverse Beam Search: Instead of greedy decoding or standard beam search, we use Diverse Beam Search with 3 beam groups, 2 beams per group (6 beams total). This mechanism penalizes beams with similar content, helping generate more diverse answers. Main hyperparameters: alpha = 0.7 (length normalization), gamma = 0.5 (diversity penalty).

4.4 A2 – Transformer Decoder

Configuration A2 replaces the LSTM Decoder with a Transformer Decoder consisting of 4 layers, 8 attention heads, d_model=512. The core differences between the two decoders:

- LSTM Decoder (A1): Processes tokens sequentially, hidden state must be passed through time. Training speed is slower because it cannot be parallelized, but has fewer parameters.
- Transformer Decoder (A2): Uses masked self-attention so it can be parallelized during training (all timesteps processed simultaneously). Cross-attention with image features and question features is more diverse (multi-head). More efficient on longer sequences.

Positional Encoding: A2 uses sinusoidal positional encoding (as in “Attention Is All You Need”) to provide positional information to the decoder, because self-attention has no inherent notion of order.

Cross-attention: Each decoder layer contains a cross-attention block after masked self-attention, taking keys and values from the encoder output (combined image patches and question features), helping the decoder “query” the encoder for information to generate the next word. This is an important difference from the simpler attention in A1.

All other training configurations (epoch, batch size, optimizer, learning rate, loss function, inference strategy) remain identical to A1, so that when comparing A1 vs A2 we can directly attribute result differences to the decoder type. This is a controlled experiment.

CHAPTER 5. METHODOLOGY – APPROACH B (MULTIMODAL PRETRAINED)

5.1 BLIP-2 Architecture and Vietnamese Processing Strategy

Approach B requires using Multimodal Pretrained models (BLIP/BLIP-2, ViLT, LLaVA, Qwen-VL, PaliGemma…). We chose BLIP-2 with the OPT-2.7b model (Salesforce/blip2-opt-2.7b) for the following reasons: (1) ready-to-use integration with HuggingFace Transformers, (2) moderate VRAM requirements (≥ 15 GB) — fitting the Tesla T4 GPU on free Colab, (3) reasonably good zero-shot quality on multiple English VQA benchmarks.

However, BLIP-2 (like most large multimodal pretrained models) is trained primarily on English data. Some models like Qwen-VL or LLaVA support multilingual but English quality still clearly dominates. Therefore, we apply a two-way translation strategy:

Step 1: Question (VI) → Google Translator → Question (EN)

Step 2: (Image, Question EN) → BLIP-2 → Answer (EN)

Step 3: Answer (EN) → Google Translator → Answer (VI)

The translation module uses the deep_translator library (a wrapper for the Google Translate API). This approach maximizes BLIP-2's capability in English, but at the cost of error accumulation through two translation steps — a weakness we will analyze in Chapter 7.

5.2 B1 – Zero-shot Inference

Configuration B1 does not train anything. The BLIP-2 model is loaded with 8-bit quantization (BitsAndBytesConfig) to fit within 15 GB VRAM of the Tesla T4, then uses the translation pipeline in section 5.1 for direct inference.

Advantages: No training time/GPU required. BLIP-2 is pretrained on billions of (image, text) pairs so it understands images very well. Flexible — can answer open-ended questions rather than being limited by a fixed vocabulary like A1/A2.

Limitations: It lacks domain knowledge — it does not know Vietnamese dish names such as “phở,” “bún bò Huế,” or “bánh mì.” It depends on translation quality, and errors accumulate through the two translation steps. Latency is also high because each query must call both the translation API and the model inference.

5.3 B2 – Fine-tuning with LoRA / PEFT

To overcome the domain knowledge limitation of B1, configuration B2 fine-tunes BLIP-2 on the training set of the Vietnamese cuisine dataset — but using LoRA instead of full fine-tuning. Reasons for choosing LoRA:

- VRAM saving: full fine-tuning BLIP-2 requires ≥ 40 GB VRAM, far exceeding the Tesla T4's capacity. LoRA + 8-bit quantization fits comfortably within 15 GB.
- Avoids catastrophic forgetting: since only ~0.13% of parameters are updated, BLIP-2's pretrained knowledge is preserved.
- Fast convergence: with low rank r=16, the number of trainable parameters is only 5.24M — trains much faster than full fine-tuning.
- Can be merged after training: no increase in inference latency.

Detailed configuration is presented in Table 5.1:

Table 5-1: Detailed configuration of model B2.

Hyperparameter	Value
Base model	Salesforce/blip2-opt-2.7b (~3.7B params)
Quantization	8-bit (BitsAndBytesConfig)
PEFT method	LoRA
LoRA rank (r)	16
LoRA alpha	32
LoRA dropout	0.05
Target modules	q_proj, v_proj của OPT-2.7B
Trainable params	5.24M / 3.75B (~0.13%)
Number of epochs	1 (see explanation in Chapter 7)

Hyperparameter	Value
Effective batch size	16 (4 × gradient accumulation)
Learning rate	1e-4 (linear schedule + warmup 10%)
Total steps	315
GPU sử dụng	Tesla T4 (15.6 GB VRAM)

Important note on number of epochs: Due to time and free GPU resource constraints on Colab, B2 was only trained for 1 epoch (315 steps). Train loss decreased from ~5.5 to 1.51, val loss reached 1.20 — indicating the model is on an effective learning trajectory but has not yet converged. This is an acknowledged major limitation and will be analyzed in detail in Chapter 7. An extended experiment with 3–5 epochs should be conducted in future versions.

CHAPTER 6. EVALUATION METHODS

The assignment requires a separate chapter analyzing evaluation methods, because VQA is a complex generation task — no single metric is sufficient. In this chapter, we present the six evaluation methods used or planned, discuss the strengths and weaknesses of each metric, and explain why multiple metrics are needed for a comprehensive view.

6.1 VQA Accuracy

VQA Accuracy is the most classic metric for VQA, officially introduced in the VQA v2 dataset. There are two variants:

- Exact match: The generated answer must exactly match the ground-truth answer after normalization (lowercasing, removing punctuation). This method is simple, but overly strict — for example, “yes” and “yes, it does” would be considered different and counted as incorrect.
- Soft accuracy (standard VQA v2): For each Q&A pair, there are 10 answers collected from 10 different annotators. The model’s accuracy is calculated as min (number of annotators providing the same answer / 3, 1). This approach is more tolerant of different but equally valid expressions.

Implementation in the project: Since our dataset only has one answer per pair (generated by Gemini), we use exact match. This is a limitation and typically gives lower scores than reality. In notebooks B1 and B2, VQA Accuracy is calculated on the 631-sample test set. A1 and A2 have not computed this metric — this will be addressed in Chapter 7.

6.2 BLEU, ROUGE-L, METEOR – N-gram Metrics

BLEU (Bilingual Evaluation Understudy): Measures n-gram precision between the generated sentence and reference, with a brevity penalty to punish overly short sentences. BLEU-1 only considers unigrams (~precision of individual words), BLEU-4 considers up to 4-grams (measuring fluency). BLEU is popular in machine translation but has drawbacks: it does not understand synonyms and depends on exact word matching.

ROUGE-L (Recall-Oriented Understudy for Gisting Evaluation): Based on the Longest Common Subsequence (LCS) between the generated sentence and reference. ROUGE-L F1 balances precision and recall. Suitable for text summarization evaluation, inherited for VQA.

METEOR: An improvement of BLEU, supporting matching by word stem, synonym (via WordNet), and chunking (contiguous phrases). METEOR usually correlates better with human evaluation. However, Vietnamese support is limited — we can still use it as the nltk.translate.meteor_score library processes at the word level.

6.3 BERTScore

BERTScore (Zhang et al., ICLR 2020) uses pretrained BERT embeddings to represent each token in the generated sentence and the reference sentence, then computes cosine similarity between the most semantically similar token pairs, and finally calculates the F1 score. Unlike BLEU or ROUGE, BERTScore can capture semantic similarity: for example, “yellow” and “slightly yellow” may still receive a high score even if they do not match exactly at the word level.

In the project, we use bert-base-multilingual-cased (mBERT) as the scorer, as it supports Vietnamese well. BERTScore F1 is calculated across all 4 configurations A1, A2, B1, B2 — this is the metric we trust most for Vietnamese data because it is objective regarding different expressions.

6.4 LLM-as-a-judge

LLM-as-a-judge is a recent evaluation method that uses a powerful LLM such as GPT-4, Gemini, or Claude as an “automatic evaluator.” A typical process is to provide the LLM with a prompt containing (the question, the ground-truth answer, and the model’s predicted answer), then ask it to assign a score from 1–5 or judge the answer as correct/incorrect with an explanation. Advantages:

- Can evaluate deep semantics, accepting diverse expressions (like humans).
- More suitable for open-ended answers than n-gram metrics.
- Can design detailed rubrics: accuracy, naturalness, appropriate length…

Limitations: depends on LLM quality, may have bias (LLMs tend to favor longer answers, or answers similar to those of the judge LLM).

Implementation status in the project: We studied this method during the design phase but could not implement it in time due to time constraints and Gemini API quota limits. This is a candidly acknowledged limitation — and also a clear development direction for the extended version. Specific implementation proposals are presented in Chapter 10.

6.5 Metric Summary and Selection

Table 6-1: Summary of evaluation methods studied.

Metric	Measures	Suitable for
VQA Accuracy	Exact match (lower-case normalized)	Short, fact-based answers
BLEU-1, BLEU-4	N-gram overlap	Long, structured answers
ROUGE-L	Longest Common Subsequence (F1)	Summarization, summary
METEOR	Synonym + word stems + chunks	Shallow semantic evaluation
BERTScore	Cosine similarity of BERT embeddings	Deep semantic similarity
LLM-judge	LLM (GPT-4 / Gemini) scores 1–5	Holistic evaluation, close to human

In Chapter 7, we will report results across multiple metrics and discuss which configuration performs well on which metric, thereby drawing more comprehensive insights than relying on a single metric.

CHAPTER 7. EXPERIMENTS AND RESULTS

This is the central chapter of the report. We will present the complete experimental results for all 4 configurations A1, A2, B1, B2; then focus on analyzing the two most important comparison pairs required by the assignment: (i) A1 vs A2 — the effect of decoder type; (ii) A vs B — modular architecture vs multimodal pretrained. Finally, error analysis by question type and transparent disclosure of limitations.

7.1 Experimental Setup

Hardware: All training and evaluation was performed on Google Colab Pro / free Colab with Tesla T4 GPU (15.6 GB VRAM, CUDA 13.0). BLIP-2 models (B1, B2) were loaded with 8-bit quantization (BitsAndBytesConfig) to save VRAM.

Software: Python 3.12, PyTorch 2.x, Transformers 4.40, PEFT (LoRA), bert-score, rouge-score, nltk (BLEU, METEOR), matplotlib (visualization), Hugging Face Hub (loading PhoBERT, ViT, BLIP-2).

Reproducibility: All random seeds are fixed at random_state=42 for data splitting steps. Full source code and checkpoints are submitted along with the dataset on Google Drive.

Table 7-1 below presents the aggregated results for all 4 configurations. The columns represent the 5 metrics introduced in Chapter 6.

Table 7-1: Aggregated results for all 4 configurations. Best result per metric highlighted in blue.

Configuration	BLEU-1	BLEU-4	ROUGE-L	METEOR	BERTScore
A1 (LSTM Dec.)	23.32	7.44	35.11	19.45	76.84
A2 (Trans Dec.)	46.64	22.26	52.37	41.86	83.00
B1 (Zero-shot)	14.35	3.73	31.05	15.11	73.72
B2 (Fine-tuned)	13.10	2.27	28.86	10.15	73.21

Important note regarding comparison: As explained in Section 3.4, A1 and A2 are evaluated on the 1,261-sample validation set (80/20 split), while B1 and B2 are evaluated on the 631-sample test set (80/10/10 split). Although data distribution is similar due to using the same random_state, the absolute numbers are only for reference when directly comparing between the two groups. Instead, we analyze trends and relationships between configurations within the same group.

7.2 Comparison A1 vs A2 – Effect of Decoder Type

This is the most important controlled experiment in the project: the same image encoder (ViT-B/16), the same text encoder (PhoBERT-base), the same dataset, the same number of epochs (30), the same optimizer/learning rate, and the same inference strategy (Diverse Beam Search). The only difference is the decoder type. Table 7.2 presents the detailed comparison:

Table 7-2: Detailed comparison of A1 (LSTM Decoder) and A2 (Transformer Decoder).

Metric	A1 (LSTM)	A2 (Trans)	Absolute Δ	Relative Δ
BLEU-1	23.32	46.64	+23.32	+100.0%
BLEU-4	7.44	22.26	+14.82	+199.2%
ROUGE-L	35.11	52.37	+17.26	+49.2%
METEOR	19.45	41.86	+22.41	+115.2%
BERTScore F1	76.84	83.00	+6.16	+8.0%
Train Loss (final)	3.28	1.89	−1.39	−42.4%
Val Loss (final)	3.40	2.40	−1.00	−29.4%

7.2.1 Key Findings

Transformer Decoder clearly outperforms on all metrics. Most notably, BLEU-1 doubled (23.32 → 46.64), BLEU-4 nearly tripled (7.44 → 22.26),

METEOR more than doubled (19.45 → 41.86). BERTScore — the most semantically meaningful metric — also improved significantly from 76.84 to 83.00 (+6.16 absolute points, or +8% relative).

Train/Val Loss is significantly lower. A2's final Train Loss reached 1.89 (vs 3.28 for A1), Val Loss reached 2.40 (vs 3.40 for A1). This means A2 has better learning capability from the data over the same number of epochs.

A2 shows slight signs of overfitting at the end. The Train Loss to Val Loss gap of A2 (1.89 vs 2.40, gap = 0.51) is larger than A1's (3.28 vs 3.40, gap = 0.12). This is actually a positive sign: A2 is powerful enough to memorize training data, while A1 has not yet achieved this. In future versions, higher dropout or data augmentation could be applied to A2 to better utilize its capacity.

7.2.2 Technical Explanation

Why does Transformer Decoder outperform? There are three main reasons:

(1) Multi-head cross-attention allows the decoder to "look" at different parts of the image and question features simultaneously from multiple "views" (8 heads). By contrast, A1 uses only a single simple spatial attention channel. This is especially helpful when the answer must combine multiple aspects from the image and question.

(2) Parallelizable training. The Transformer processes the entire output sequence in parallel during training (teacher forcing with causal mask), while LSTM must process sequentially. This not only increases training speed but also improves gradient stability — because Transformer gradients do not vanish or explode over time like LSTM.

(3) Self-attention in the decoder allows the currently generated token to "look back" at all previous tokens with selectable attention weights, instead of relying only on a compressed hidden state as in LSTM.

Conclusion of this experiment: Under the same training resources and with the same image+text encoder, the Transformer Decoder consistently and significantly outperforms the LSTM Decoder. This result is consistent with the general trend in Natural Language Processing and Computer Vision, where Transformer-based architectures are gradually replacing RNN/LSTM models in most generation tasks. In real-world projects, if computational resources permit, the Transformer Decoder should be prioritized.

7.3 Comparison A vs B – Modular Architecture vs Multimodal Pretrained

The second comparison pair is between the two approaches: (i) A — building a modular architecture trained end-to-end on the specialized dataset; (ii) B — fine-tuning a massive pretrained multimodal model on the same dataset (or using zero-shot).

Surprising Finding: A2 outperforms both B1 and B2 on all n-gram metrics (BLEU, ROUGE, METEOR) as well as BERTScore. Specifically, A2's BLEU-1 (46.64) is about 3.2 times B1 (14.35) and 3.6 times B2 (13.10). A2's BERTScore F1 is about 10 points higher than B1/B2. This is an interesting result that goes against the expectation that "pretrained models are always better."

7.3.1 Why Does the From-Scratch Model Outperform the Pretrained Model?

There are four main reasons explaining this phenomenon:

(1) Specialized data with a small vocabulary. Our dataset only contains questions/answers about Vietnamese food, with a vocabulary of only 693 words. The from-scratch model (A2) only needs to learn the image→answer mapping over this small space, while BLIP-2 (B1, B2) must operate on a vast English language space — causing losses when converting through translation.

(2) The bidirectional translation pipeline in Approach B accumulates errors. Each translation step (Vi→En and En→Vi) can alter meaning or cause content loss. This is especially true for Vietnamese culinary terms such as "bún riêu," "bánh khọt," and "bún đậu mắm tôm," which Google Translate often translates incorrectly or inconsistently transliterates.

(3) BLIP-2 lacks domain knowledge about Vietnamese cuisine. In the B1 demo, when asked "What color is the bánh mì?" the model answered with a generic Wikipedia-style response rather than inspecting the actual image. This shows the model does not have specific knowledge about Vietnamese dishes.

(4) B2 was only fine-tuned for 1 epoch — not yet converged. This is an acknowledged major limitation. Train Loss decreased from ~5.5 to 1.51 in only 315 steps, val loss reached 1.20 — indicating the model is learning effectively, but if trained for an additional 3–5 epochs, results would almost certainly improve significantly.

An important highlight of B2: although BLEU/ROUGE/METEOR scores are lower than B1, VQA Accuracy increased 4× (0.32% → 1.27%), and recognition

accuracy rose from 0.0% to 4.5%. This proves that LoRA fine-tuning actually taught the model Vietnamese dish names. The final demo of B2 correctly identified "Bún riêu" — something B1 could not do. With more training epochs and better handling of translation, B2 could fully surpass B1.

7.3.2 Lessons Learned

Pretrained is not a universal solution. On specialized data domains with distributions far from pretraining data (such as Vietnamese, Vietnamese cuisine), a fully trained from-scratch model can outperform massive pretrained models — especially when the large models are not fully fine-tuned.

Pretraining language selection matters. If the goal is Vietnamese VQA, instead of BLIP-2 (primarily English) with a complex translation pipeline, consider models with direct Vietnamese support such as Qwen-VL, LLaVA-NeXT, or PaliGemma — these are clear improvement directions for future projects.

7.4 Error Analysis by Question Type

Aggregating a single number for each configuration hides many detailed lessons. Table 7.4 presents VQA Accuracy of B1 and B2 broken down by question type (only B1, B2 have VQA Accuracy due to the explained limitations):

Table 7-3: VQA Accuracy by question type for B1 and B2.

Question types	Samples	B1 Acc.	B2 Acc.	Improvement
Yes/No	409	0.2%	0.5%	+0.3%
Recognition	132	0.0%	4.5%	+4.5%
Color	46	2.2%	0.0%	−2.2%
Spatial	19	0.0%	0.0%	0.0%
Counting	1	0.0%	0.0%	0.0%
Other	24	0.0%	0.0%	0.0%

Important observations: “Recognition” questions (“What dish is this?”) show the clearest improvement after fine-tuning (0.0% → 4.5%). This confirms that LoRA actually taught the model Vietnamese dish names. In contrast, Yes/No and Color questions barely improved — suggesting that further fine-tuning on these question types or separate post-processing is needed.

7.5 Limitations and Transparent Disclosure

To help readers properly evaluate the value of this report, we list the limitations candidly:

Inconsistent data split ratios between Approach A (80/20) and Approach B (80/10/10). The assignment requires 80/10/10. This is a procedural error. Impact: A1/A2 and B1/B2 figures cannot be directly compared in a strict mathematical sense; only relative trends can be compared.

VQA Accuracy has not been calculated for A1, A2. This is the primary VQA metric. In future versions, this needs to be computed for A1, A2 on the same 631-sample test set for direct comparison with B1, B2.

The test set does not fully satisfy the requirement. The assignment requires “≥ 50 manually prepared test samples, with no overlap with train images.” Currently, the test set is split randomly at the (image, question, answer) level, meaning the same image can appear in both train and test. Improvement: split by image (group-aware split) and collect additional images outside the original dataset.

LLM-as-a-judge has not been implemented. The method was studied, with plans to use Gemini 2.5 Flash or GPT-4 to score 1–5 for each prediction-reference pair, but was not completed due to time constraints and API quota limits.

B2 was only trained for 1 epoch — not yet converged. Train Loss decreased from 5.5 to 1.51 but there is still room for improvement. Training an additional 3–5 epochs is needed for fairer conclusions about fine-tuned Approach B.

The translation pipeline for Approach B accumulates errors. In particular, for Vietnamese culinary terms, Google Translate is inconsistent. One translation bug recorded in the B2 notebook was that the sentence “Món này có chả ăn kèm phải không?” could not be translated and had to fall back to the original Vietnamese — leading to invalid output.

CHAPTER 8. Advanced Solution – Preparing Data for RL/DPO

The assignment (section 7) encourages exploring advanced solutions for improving model quality, including Reinforcement Learning (PPO/DPO/RLHF) with rewards based on VQA Accuracy or BERTScore, and requires preference data ≥ 100 pairs. Within the time constraints, we completed the data preparation step — the foundational step for any RL pipeline — but did not have time to implement the actual DPO/PPO training step.

8.1 Preference Data Collection Strategy

Preference data in DPO/RLHF requires three components for each sample: (prompt, chosen, rejected). We built an automatic collection pipeline as follows:

For each (image, question) pair in the evaluation set, run the SFT model (already trained, A1 or A2) to generate a predicted answer.

Compare the model's answer with the ground-truth answer using an overlap metric (number of common tokens / length of the longer sentence).

If overlap < 0.5, mark it as “rejected” (incorrect answer); the ground-truth answer is “chosen.” Save the pair to the preference dataset.

If overlap ≥ 0.5 → consider the model's answer close to ground-truth, do not create a preference pair for this sample.

8.2 Collection Results

Table 8-1: Preference data collection results from SFT models A1, A2.

SFT Configuration	Samples Processed	Preference Pairs Collected
A1 (LSTM Dec.)	1.261	785
A2 (Trans Dec.)	1.261	480
Total	2.522	1.265

Interesting Analysis: A1 (LSTM Decoder) generates more preference pairs than A2 (785 vs 480) — because A1 makes more errors than A2. This reconfirms the results in Chapter 7: A2 outperforms A1. Notably, the total number of

preference pairs (1,265) far exceeds the minimum requirement in the assignment (100 pairs), creating a sufficiently large foundation for DPO training in the future.

8.3 Future DPO Implementation Direction

With the preference data available, specific DPO implementation direction:

- Use the trl library (Hugging Face) to set up DPO Trainer; reference model = SFT model A2; policy model = a copy of A2 with a trainable head.
- Loss function: DPO loss with $\beta = 0.1$ — prevents the policy model from deviating too far from the reference.
- Evaluation: compare A2-SFT vs A2-DPO on the same test set, using both automatic metrics (VQA Acc, BERTScore) and human evaluation (each team member scores 50 samples, computing Inter-Annotator Agreement).

Expected: DPO will improve answer quality in contexts where SFT has not performed well — especially spatial questions, counting questions, and yes/no questions with complex context.

CHAPTER 9. DEMO AND REAL-WORLD APPLICATIONS

9.1 Demo Interface

Each configuration A1, A2, B1, and B2 has an interactive demo integrated into its corresponding notebook. The demo uses google.colab.files.upload() to let users upload an image, then enter a Vietnamese question and receive the model's answer. Two special commands support interaction: "new" (switch to a new image) and "thoat" (end the demo session).

Inference – Diverse Beam Search: Both A1 and A2 use Diverse Beam Search with 3 groups × 2 beams (6 beams total) to increase answer diversity. Hyperparameters: max_len=15 tokens, alpha=0.7 (length normalization), gamma=0.5 (diversity penalty). B1 and B2 use BLIP-2's default generation with max_new_tokens=20.

9.2 Real Output Examples

Below are some illustrative examples on the uploaded "Bún riêu" image in the demo:

A1 (LSTM Decoder): When asked "What is this dish?" → answered "bún bò huế" (incorrect); when asked "What color is the broth?" → "green" (incorrect). A1 often makes serious errors and cannot distinguish visually similar dishes.

A2 (Transformer Decoder): When asked "What dish is this?" → answered "bánh bèo" (incorrect for the bún riêu image); when asked "What color is the broth?" → "reddish orange broth" (correct); when asked "What ingredients are on top?" → "green onion and sliced red chili" (very accurate). The answers are more natural and detailed than A1's.

B2 (BLIP-2 + LoRA): When asked "What dish is this?" → answered "Bún riêu" (correct!). This is the clearest proof of the effectiveness of LoRA fine-tuning: the model learned Vietnamese dish names that B1 zero-shot could not identify.

Interesting observation: although A2 is wrong in overall dish recognition ("bánh bèo" instead of "bún riêu"), its detailed answers about ingredients and color are very accurate. This suggests that A2 can "see" well but sometimes confuses the dish label — possibly due to dataset bias or insufficient data for visually similar dishes.

9.3 Real-World Applications

The Vietnamese VQA system for cuisine has many potential real-world applications:

- Tourism: a mobile application helping tourists identify Vietnamese dishes in restaurants and eateries — especially useful for international visitors or those new to Vietnamese cuisine.
- Cultural education: creating interactive experiences in culinary culture museums, helping visitors learn about traditional dishes through natural question-and-answer interaction.
- Virtual assistant for visually impaired users: combined with a camera and TTS, it allows visually impaired users to “ask” about the dish in front of them — learning the dish name, ingredients, and color.
- Restaurant marketing: a website chatbot allowing customers to upload menu images and ask about dishes in Vietnamese — enhancing customer experience and reducing staff workload.

CHAPTER 10. CONCLUSION

10.1 Summary

This report presents a complete project on Vietnamese Visual Question Answering for the Vietnamese culinary domain. We built a dataset of 6,303 QA pairs by combining images from Hugging Face Hub with questions auto-generated by Gemini 2.5 Flash; implemented 4 model configurations covering both approaches required by the assignment; studied and analyzed 6 evaluation methods; and drew meaningful technical lessons through two controlled comparison pairs.

Experimental results show: (1) Transformer Decoder outperforms LSTM Decoder on all metrics when other components remain the same — consistent with the general trend in modern NLP/CV; (2) The from-scratch model (A2) can outperform pretrained models (B1, B2) on specialized datasets, especially when the pretrained model must go through a translation pipeline and has not been fully fine-tuned; (3) Fine-tuning with LoRA, despite only 1 epoch of training, demonstrated clear effectiveness through a 4× improvement in VQA Accuracy and notably in recognizing Vietnamese dish names.

10.2 Contributions

- An end-to-end pipeline for building a Vietnamese VQA dataset using LLM (Gemini) as a virtual annotator — saving many hours of manual labeling.
- A controlled comparison between LSTM Decoder and Transformer Decoder on the same image+text encoder and dataset — conclusion: Transformer Decoder is clearly superior.
- Implementation of Diverse Beam Search in inference to increase answer diversity — improving demo quality through direct visual comparison.
- A LoRA fine-tuning pipeline for BLIP-2 on Tesla T4 GPU (15.6 GB VRAM) — demonstrating the ability to train large models on limited resources.
- A preference dataset of 1,265 pairs ready for DPO/RLHF experimentation in future versions.

10.3 Limitations

As transparently presented in Section 7.5, the project has the following limitations:

- Inconsistent data split ratios between notebooks (A: 80/20 vs B: 80/10/10).
- VQA Accuracy has not been calculated for A1 and A2.

- No manually prepared test set of ≥50 samples with images not overlapping with train.
- LLM-as-a-judge and human evaluation have not been implemented.
- B2 has only been trained for 1 epoch and has not fully converged.
- The Vi↔En translation pipeline has accumulated errors — especially with Vietnamese culinary terminology.

10.4 Future Development Directions

Based on the identified limitations, we propose the following specific improvement directions:

Standardize the 80/10/10 split ratio at the image level (group-aware split), add a manually prepared test set of ≥ 50 samples with images independently collected from the web or self-photographed.

Add VQA Accuracy for A1, A2 on the same 631-sample test set for fair direct comparison with B1, B2.

Implement LLM-as-a-judge: use Gemini 2.5 Pro or GPT-4o with a detailed rubric prompt (scoring 1–5 on 3 criteria: accuracy, naturalness, appropriate length).

Train B2 for 5–10 epochs for fair conclusions about fine-tuned Approach B.

Experiment with Qwen-VL or PaliGemma — multimodal models with direct Vietnamese support, eliminating the translation pipeline.

Implement DPO with the existing 1,265 preference pairs, comparing A2-SFT vs A2-DPO.

Expand the dataset to 100+ Vietnamese dishes with 5+ questions/image, combined with data augmentation (flipping/rotating/cropping images, paraphrasing questions).

CHAPTER 11. PLANT DISEASE DETECTION USING EFFICIENTNETB0

11.1 Problem Statement

Vietnam is a country with an important agricultural economy — contributing more than 12% of national GDP. Plant diseases are one of the leading causes of crop failure and significant economic losses for farmers. However, manual disease diagnosis requires agricultural experts — who are not always available, especially in remote rural areas — and consumes considerable time and cost.

Problem: Build a Deep Learning model capable of classifying 38 disease types / healthy states across 14 common crop species from a single leaf image.

Input: An RGB image of a plant leaf (taken from a phone or camera).

Output: The disease name or health status (e.g., "Tomato – Late blight," "Apple – Black rot," "Healthy"), along with the confidence score (probability).

11.2 Why This Problem?

We propose this problem for 5 main reasons: (1) high practical applicability in the context of Vietnam as an agricultural country, immediately applicable in practice; (2) the publicly available PlantVillage dataset on Kaggle with over 54,000 labeled images (CC BY 4.0); (3) well-suited for Transfer Learning techniques — one of the foundational techniques in modern Deep Learning; (4) high deployability due to the lightweight model size (EfficientNetB0 only 5.3M params), suitable for mobile applications for farmers; (5) clear evaluation metrics: Accuracy, F1-score, Confusion Matrix.

11.3 Proposed Solution – Transfer Learning with EfficientNetB0

We use transfer learning with the EfficientNetB0 model pretrained on ImageNet. We chose EfficientNetB0 instead of VGG/ResNet because it offers the best balance between accuracy and computational cost thanks to "compound scaling" — simultaneously scaling depth, width, and resolution by an optimal ratio.

Table 11-1: Comparison of popular CNN architectures on ImageNet

Model	Top-1 Acc (ImageNet)	Parameters	FLOPs
VGG16	71.3%	138M	15.5B
ResNet50	74.9%	25M	4.1B
EfficientNetB0	77.1%	5.3M	0.39B
EfficientNetB7	84.4%	66M	37B

Specific Architecture: ImageNet pretrained EfficientNetB0 → GlobalAveragePooling2D → BatchNormalization → Dense(256, ReLU) → Dropout(0.4) → Dense(38, Softmax). Total ~4.4 million trainable parameters.

11.4 Two-Phase Training Process

This is the standard strategy for Transfer Learning on specialized datasets:

Phase 1 – Train Classification Head: Freeze the entire EfficientNetB0 backbone, only train the new classification head (Dense + Dropout + Softmax). Purpose: avoid destroying pretrained features in the early stage when gradients are large. Hyperparameters: 15 epochs, learning rate = 1e-3, optimizer = Adam, callbacks: EarlyStopping (patience=5), ReduceLROnPlateau (factor=0.5, patience=3).

Phase 2 – Fine-tuning: Unfreeze the last 40 layers of the backbone, reduce learning rate by 10× to 1e-4 to avoid destroying pretrained knowledge. Purpose: allow low/mid-level layers to be fine-tuned to fit the plant leaf domain. Number of epochs: 20.

11.5 Data Augmentation

To improve generalization and simulate real-world image capture conditions, we apply data augmentation only on the training set: rotation_range=30°, width/height_shift=±15%, shear_range=0.15, zoom_range=0.2, horizontal_flip=True, brightness_range=[0.7, 1.3]. The val and test sets retain original images for objective evaluation.

11.6 Experimental Configuration and Results

PlantVillage Dataset: 54,305 images, 38 classes, 14 crop species (Apple, Tomato, Grape, Corn, Potato, Cherry, Peach, Pepper, Strawberry, Orange, Squash, Blueberry, Raspberry, Soybean). Train/val/test split at 70/15/15 ratio: 37,997 / 8,129 / 8,179.

Hardware: Google Colab GPU Tesla T4 (15.6 GB VRAM). Total training time: ~5 hours for both Phase 1 and Phase 2.

Final results on the test set:

Table 11-2: EfficientNetB0 model results on the PlantVillage test set.

Metric	Value
Test Accuracy	98.84%
Test Top-5 Accuracy	100.00%
Test Loss	0.0373
Number of predicted classes	38
Number of test images	8.179
Total parameters	4,392,393
Best Val Accuracy (Phase 2)	97.79%

Analysis: Accuracy of 98.84% across 38 classes with 8,179 test images is an excellent result. Top-5 Accuracy reaches 100% — meaning the correct answer always appears in the top 5 most probable predictions. This has practical significance: for a farmer-facing application, the system can always display the top-5 diagnoses for users to choose from when top-1 is uncertain.

11.7 Grad-CAM – Model Explanation

To ensure that the model learns the correct patterns (the diseased regions on the leaves) rather than background artifacts (such as image background or lighting conditions), we applied Grad-CAM (Selvaraju et al., ICCV 2017). Grad-CAM computes the gradients of the output class with respect to the final feature map (top_conv of EfficientNetB0), generating a heatmap that highlights the image regions with the greatest influence on the model's decision.

From the Grad-CAM results, the heatmap typically focuses on the diseased areas of the leaves (brown spots, discoloration, lesions), indicating that the model is indeed "looking" at the correct regions. This serves as an important validation of the system's reliability, especially when deployed for non-expert users.

11.8 Web Demo with Gradio

We built a demo interface using Gradio — allowing users to upload any plant leaf image and receive results immediately. The interface consists of 3 parts: (1) image upload frame, (2) top-5 prediction table displayed as a chart with probabilities, (3) Grad-CAM heatmap display to illustrate which regions the model is attending to. The demo can be accessed via Gradio's public URL (Colab tunnel) or deployed to Hugging Face Spaces for permanent hosting.

11.9 Analysis and Conclusion for Part 2

Why does the solution work well? There are four main factors: (1) Transfer Learning from ImageNet — low-level features (edges, texture, color) learned are very useful for detecting disease spots; (2) Two-phase training avoids destroying pretrained features in the early stage; (3) Data Augmentation makes the model robust to various shooting angles and lighting conditions; (4) Grad-CAM confirms the model learns the correct disease regions.

Development directions: (i) Convert to TFLite to run directly on farmers' phones — no internet required; (ii) Expand dataset with images captured in real Vietnamese fields (more natural lighting and background conditions compared to PlantVillage studio images); (iii) Integrate Object Detection (YOLOv8) to localize disease regions across the entire plant; (iv) Few-Shot Learning to recognize new diseases not in the training set; (v) Edge AI: deploy on Raspberry Pi mounted on drones flying over fields for automated monitoring.

REFERENCES

[1] Vaswani, A., Shazeer, N., Parmar, N., et al. (2017). Attention Is All You Need. In Advances in Neural Information Processing Systems (NeurIPS), 30.

[2] Dosovitskiy, A., Beyer, L., Kolesnikov, A., et al. (2020). An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. arXiv:2010.11929.

[3] Nguyen, D. Q., & Tuan Nguyen, A. (2020). PhoBERT: Pre-trained Language Models for Vietnamese. In Findings of EMNLP 2020.

[4] Li, J., Li, D., Savarese, S., & Hoi, S. (2023). BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models. In ICML 2023.

[5] Hu, E. J., Shen, Y., Wallis, P., et al. (2021). LoRA: Low-Rank Adaptation of Large Language Models. arXiv:2106.09685.

[6] Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory. Neural Computation, 9(8), 1735–1780.

[7] Antol, S., Agrawal, A., Lu, J., et al. (2015). VQA: Visual Question Answering. In ICCV 2015.

[8] Goyal, Y., Khot, T., Summers-Stay, D., Batra, D., & Parikh, D. (2017). Making the V in VQA Matter: Elevating the Role of Image Understanding in Visual Question Answering. In CVPR 2017.

[9] Papineni, K., Roukos, S., Ward, T., & Zhu, W.-J. (2002). BLEU: a Method for Automatic Evaluation of Machine Translation. In ACL 2002.

[10] Lin, C.-Y. (2004). ROUGE: A Package for Automatic Evaluation of Summaries. In Workshop on Text Summarization, ACL 2004.

[11] Banerjee, S., & Lavie, A. (2005). METEOR: An Automatic Metric for MT Evaluation with Improved Correlation with Human Judgments. In ACL Workshop on Intrinsic and Extrinsic Evaluation Measures for MT.

[12] Zhang, T., Kishore, V., Wu, F., Weinberger, K. Q., & Artzi, Y. (2020). BERTScore: Evaluating Text Generation with BERT. In ICLR 2020.

[13] Rafailov, R., Sharma, A., Mitchell, E., et al. (2023). Direct Preference Optimization: Your Language Model is Secretly a Reward Model. In NeurIPS 2023.

[14] Vijayakumar, A. K., Cogswell, M., Selvaraju, R. R., et al. (2018). Diverse Beam Search: Decoding Diverse Solutions from Neural Sequence Models. In AAAI 2018.

[15] Tan, M., & Le, Q. V. (2019). EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. In ICML 2019.

[16] Selvaraju, R. R., Cogswell, M., Das, A., et al. (2017). Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization. In ICCV 2017.

[17] Hughes, D. P., & Salathé, M. (2015). An open access repository of images on plant health to enable the development of mobile disease diagnostics. arXiv:1511.08060.

[18] Liu, H., Li, C., Wu, Q., & Lee, Y. J. (2024). Visual Instruction Tuning (LLaVA). In NeurIPS 2023.

[19] Bai, J., Bai, S., Yang, S., et al. (2023). Qwen-VL: A Versatile Vision-Language Model for Understanding, Localization, Text Reading, and Beyond. arXiv:2308.12966.

[20] TuyenTrungLe (2023). Vietnamese Food Images Dataset. Hugging Face Hub: TuyenTrungLe/vietnamese_food_images.

[21] Google DeepMind (2025). Gemini 2.5 Flash – Multimodal Generative AI Model. https://deepmind.google/technologies/gemini/.